

American University of Armenia

---

College of Science and Engineering

Artificial Intelligence Project

# **Sudoku Solver Using CSP**

Team: Levon Titanyan, Artsvik Avetisyan, Ani  
Gharibyan

Fall, 2024

# Abstract

Sudoku is a logic-based puzzle game that was created in 1979 by Howard Garns and is loved by players all throughout the world. (*The History of Sudoku | Play Free Sudoku, a Popular Online Puzzle Game*, n.d.) It has simple rules and can have different levels of complexity. Despite its straightforward nature, solving Sudoku puzzles, especially those of greater difficulty, often requires traversing a huge search space, which can overwhelm human solvers. Sudoku's computational challenge makes it an ideal choice for artificial intelligence approaches.

This project focuses on solving Sudoku problems with Constraint Satisfaction Problem (CSP) approaches. Our approach involves using fundamental methods like backtracking and improving them with sophisticated techniques like forward checking, K consistency, and heuristics like Minimum Remaining Values (MRV). The goal is to determine the performance and efficiency of these strategies for solving Sudoku puzzles of varying difficulty levels, as well as their scalability and effectiveness. Through this study, we hope to shed light on the capabilities of CSP approaches and their applicability to similar combinatorial issues.

**Keywords:** Sudoku, Constraint Satisfaction Problem, backtracking, forward checking, K consistency, Minimum Remaining Values (MRV), heuristic algorithms



# Contents

## **Abstract**

### **1. Introduction**

- 1.1 Problem setting and description
- 1.2 Chosen technique
- 1.3 Reason for choosing the topic and its goal
- 1.4 Paper structure

### **2. Literature Review**

- 2.1 Existing Solutions

### **3. Methodology**

- 3.1 Background
- 3.2 Techniques Discussion and Comparison
- 3.3 Found Solution

### **4. Conclusion**

- 4.1 Results
- 4.2 Conclusion
- 4.3 Future Work



# Chapter 1

## Introduction

### 1.1 Problem Setting and Description

Sudoku (see Figure 1.1) is a logic-based puzzle game invented by Howard Garns in 1979 and first published in Dell Pencil Puzzles and Word Games magazine. The puzzle comprises a 9x9 grid divided into nine 3x3 subgrids. The game aims to fill in missing numbers so that every row, column, and subgrid contains all numbers from 1 to 9, with no repetition. The game is accessible as the rules are straightforward; besides, there are different difficulty levels: from fundamental problems to highly challenging ones, the game attracts people of any age and background. News and interesting problems about the game are being published every day on the internet, and the game appeals to millions of people due to its logic-based nature. Despite the simplicity of understanding the problem, the solution requires advanced techniques, careful planning, and the ability to navigate through complex logical structures. These characteristics make sudoku an impressive subject in Computational Studies, particularly in Artificial Intelligence.

|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A |   |   | 3 |   | 2 |   | 6 |   |   |
| B | 9 |   |   | 3 |   | 5 |   |   | 1 |
| C |   |   | 1 | 8 |   | 6 | 4 |   |   |
| D |   |   | 8 | 1 |   | 2 | 9 |   |   |
| E | 7 |   |   |   |   |   |   |   | 8 |
| F |   |   | 6 | 7 |   | 8 | 2 |   |   |
| G |   |   | 2 | 6 |   | 9 | 5 |   |   |
| H | 8 |   |   | 2 |   | 3 |   |   | 9 |
| I |   |   | 5 |   | 1 |   | 3 |   |   |

Figure 1.1: Initial state of Sudoku game

## 1.2 Chosen Technique

Sudoku can be classified as a Constraint Optimization Problem (CSP), as it has constraints and computational challenges, which will be discussed further in the paper. CSP problems involve assigning values to variables while not violating any constraint and solving a Sudoku problem will be an ideal opportunity and chance for developing and evaluating AI techniques to be able to solve more complex real-world problems. Sudoku has a rich environment for testing and implementing different fundamental algorithms in AI, such as backtracking, forward checking, and K consistency. Adding heuristics to these approaches allows us to analyze and refine various problem-solving techniques. One effective heuristic is Minimum Remaining Values (MRV), which will be included in our methods. It selects the variable with the fewest legal values left in its domain, narrowing down possibilities and accelerating the solving process of challenging puzzles.

## 1.3 Reason for choosing the topic and its goal

The uniform structure and diversity of difficulty levels make the game an adaptable and flexible resource for evaluating various algorithms across various challenges. The method developed in the project can be extended and used for solving other optimization problems with similar structures, showing how flexible and helpful the chosen approach is. In this project, we aim to create a powerful Sudoku solver using CSP techniques. We will implement foundational techniques such as backtracking, forward checking, and more. This project evaluates the performance, efficiency, and scalability of various approaches to puzzles of different difficulty levels. With this study, we aim to improve and expand CSP strategies, showing how they can be used to solve challenging problems beyond Sudoku.

## 1.4 Paper Structure

The layout of this paper is as follows: the Literature Review highlights the advantages and disadvantages of the various Sudoku-solving strategies and research on Constraint Satisfaction Problems (CSP). Several methods and techniques used in this work are designed and implemented in detail in the Methods section. The Experiments and Results section provides a detailed analysis of the results, explains the test environment, and assesses how well the approaches performed. Lastly, the conclusion highlights the key takeaways from the investigation, talks about the effects of the findings, and makes recommendations for possible future research topics.

# Chapter 2

## Literature Review

Many researchers admit that CSP is the primary strategy for solving Sudoku puzzles, using each cell as a variable and applying constraints across rows, columns, and sub-grids. Multiple methods have been studied for efficiently solving Sudoku as a CSP. In this part of the paper, we will dive deeper into several studies that have been done before and will figure out what methods were used, whether they were successful or not and what conclusions were made.

### 2.1 Existing Solutions

In one of the researches done so far, the researcher shows the significance of variable and value selection heuristics in backtracking search. The authors tried several combinations of solutions that included variable selection heuristics and value selection heuristics. As a variable selection heuristics, they selected such heuristics as Minimum Domain Size (MID), which determines the variable with the smallest remaining domain, and Maximum Domain Size (MAD), which selects the variable with the largest remaining domain—also incorporated that with Value Selection Heuristics, such as Smaller Value of Domain (SVI), that chooses the smallest value in the domain, Greater Value of Domain (GV), that determines the largest value in the domain, Average Value of Domain (AV), that chooses the value closest to the average of the domain, Greater Than Average Value (GAV), that determines the smallest value more significant than the average of the domain. In conclusion, they found that Minimum Domain Size (MiD) heuristics effectively reduce the search space, outperforming Maximum Domain Size (MaD) in solving difficult puzzles (Crawford & Gervet, 2009).

In another study, backtracking was the primary technique chosen to solve Sudoku with CSP, as authors knew that it worked efficiently in related combinatorial problems such as the N-queen problem. The researchers applied backtracking search to the Sudoku domain and added forward checking, which meets constraints by deleting invalid values from neighboring variables domains after assigning a value. They planned on including methods such as Minimum Remaining Value but had some difficulties with running time and could not implement it completely. Their program was tested on both specified and randomly generated Sudoku boards created with some puzzle generators. They got decent results, but when they decided to include a random hill-climbing technique, their solver performed poorer than other solvers. They discovered that solution time varied considerably according to the puzzle difficulty and grid size that they were choosing. The authors concluded that their CSP-based technique needed additional optimization. They expected that MRV or other variable selection algorithms would perform better by minimizing search space and conflicts (Christensen & Emerson, 2006).

In another research, a group of students was assigned to solve Sudoku with CSP using different methods. The methods used include Constraint Propagation (Implementing the AC3 algorithm to make arc consistent, which reduces the domain of each variable/cell by removing invalid values based on constraints) and Backtracking Search, which systematically explores possible solutions, backtracking when conflicts with constraints arise. The program they created was tested on two sets of Sudoku puzzles. Euler.txt (puzzles from Project Euler) and Magictour.txt (more difficult puzzles). In conclusion, they managed to show that combining arc consistency (AC3) with backtracking search ensures an efficient and systematic approach to solving Sudoku puzzles, even for more challenging cases (Eck, 2021)

Another research shows how the author developed a hybrid algorithm to solve Sudoku by combining Constraint Programming (CP) and Stochastic Search techniques, specifically Simulated Annealing (SA). The method that he developed worked in two stages. The first was applying Constraint Programming (CP) to fill as many cells as possible using logical deduction. As a second step, the Simulated Annealing (SA) was applied to complete the remaining grid, optimizing by swapping values within boxes to reduce constraint violations and satisfy all rules. The hybrid approach was tested on Sudoku grids of different sizes (3x3, 9x9, and more) and compared against Simulated Annealing (SA) alone. The results showed that the hybrid method outperformed simple Simulated Annealing, particularly on more difficult puzzles. The CP step effectively reduced the search space, making it easier for SA to solve the remaining grid. In conclusion, the author stated that combining these two methods creates a more efficient solver, as CP simplifies the problem while SA navigates the reduced search space (Lewis, 2007).

In conclusion, many methods like backtracking, forward checking, and arc consistency have proved effective. Some researchers and studies have also worked on showing hybrid approaches to enhance the effectiveness of more difficult Sudoku Puzzles. They tried to combine logical deduction with stochastic optimization, like Simulated Annealing, as well as go with some more interesting heuristics, such as MID and MAD; these methods collectively demonstrate that solving Sudoku efficiently requires reducing the search space, applying constraints effectively, and leveraging heuristics to navigate complex solution spaces. This gives us a reason to explore hybrid approaches further to figure out the best of them all.

# Chapter 3

## Methodology

### 3.1 Background

This project includes CSP algorithms and heuristics, whose definitions and usage will be discussed in this chapter. Since the time and space complexities are different, it is necessary to test each algorithm and heuristic separately and appropriately while not forgetting about the combinations of heuristics and algorithms because they are apparent. In most cases, the combinations of heuristics and algorithms become the best and most fantastic solution to the problems. This chapter also compares Breadth First Search, Depth First Search, Iterative Deepening Search, and CSP techniques to prove one more time that CSP techniques are the most suitable ones for Sudoku.

### 3.2 Techniques Discussion and Comparison

The first step that needed to be done was to prove that CSP techniques are the best ones compared to other search methods. The analysis of Breadth First Search, Depth First Search, and Iterative Deepening Search on randomly generated Sudoku found that the best among these three search methods is Depth First Search with minimum time complexity (0.0093 seconds in the best-case scenario). The next step was to test different algorithms, heuristics, and their combinations to understand which way we should choose to continue working. The first comparison we made was checking the performances of MRV, LCV heuristics, their combined version, and the AC-3 algorithm in different difficulty levels. The result was that in 3 difficulty levels (easy, medium, hard), the MRV heuristic separately performed best with steps and time complexity (see Figure 3.1). The following analysis was conducted using MRV, Degree heuristics, AC-3 algorithm, and Forward Checking approach at different difficulty levels. The surprising finding from this test was that the Degree heuristic failed to complete solving Sudoku puzzles at the hard difficulty level. However, when looking at the time spent on solving puzzles at easy and medium levels, the time that the Degree Heuristic took might suggest that the hard-level Degree heuristic would fail. In this comparison, again, it is seen that the MRV heuristic behaves the best, but the AC-3 algorithm follows the MRV heuristic. Then, we tried to

|    | Heuristic | Difficulty | Time Taken (s) | Solved | Total Steps |
|----|-----------|------------|----------------|--------|-------------|
| 0  | mr        | easy       | 0.0112         | True   | 6163        |
| 2  | mr        | hard       | 0.0156         | True   | 8369        |
| 1  | mr        | medium     | 0.0157         | True   | 7951        |
| 9  | ac3       | easy       | 0.0166         | True   | 8923        |
| 10 | ac3       | medium     | 0.0192         | True   | 10437       |
| 3  | lcv       | easy       | 0.0194         | True   | 10744       |
| 6  | combined  | easy       | 0.0200         | True   | 11528       |
| 4  | lcv       | medium     | 0.0230         | True   | 12691       |
| 7  | combined  | medium     | 0.0278         | True   | 15539       |
| 11 | ac3       | hard       | 0.0316         | True   | 14558       |
| 8  | combined  | hard       | 0.0372         | True   | 19845       |
| 5  | lcv       | hard       | 0.0471         | True   | 26596       |

Figure 3. 1

understand whether adding MRV and LCV heuristics would change the time complexity of the AC-3 algorithm. Still, the results showed that they made the algorithm misbehave at every difficulty level (see Figure 3.2). The MRV heuristic remained the leading approach to solving Sudoku puzzles at the easy and medium difficulty levels. Still, when analyzing the hard difficulty level, AC-3 outdid the MRV heuristic as expected. At the easy level, the AC-3 algorithm was the worst solution for solving puzzles; at the medium level, it came to the 2nd place; and at the hard level, as is already mentioned, the AC-3 algorithm was the best solution. After understanding the difference between several approaches, the other test that needed to be done was understanding how time complexity was added when changing the difficulty levels; the outcome was that the difference was not that much. It is obvious that when difficulty level changes, the empty cells become more and it needs a little bit more time to fill the empty cells, which logically implies that a little difference is entirely normal. Last but not least, when analyzing the difference among time complexities of different techniques at an easy level, the difference was not that much, which implies that all of the methods can be interchangeably used when solving sudoku puzzles.

### 3.3 Found Solution

The analysis of various approaches to solving Sudoku puzzles has demonstrated that the two most-effective methods, depending on the difficulty level of the puzzle, are the MRV heuristic and the AC-3 algorithm. The MRV heuristic generally performed better than others in terms of steps and time complexity for easy and medium difficulty levels. This proved that it is the best and most efficient approach for these cases. At the hard difficulty level, however, the AC-3 emerged as the best solution and exceeded the MRV heuristic. While the MRV showed a better performance in many cases, the adaptability of the AC-3 algorithm-its capabilities in terms of higher efficiency at tougher difficulty levels-highlights its robustness in tackling complex constraints. Therefore, these findings tell us that the combination of the MRV heuristic for solving easier puzzles and employing the AC-3 algorithm for solving harder ones gives a robust solution framework for solving Sudoku puzzles.

|   |           |             |            |                |        |
|---|-----------|-------------|------------|----------------|--------|
| ( | Grid Size | Heuristic   | Difficulty | Time Taken (s) | Solved |
| 0 | 9         | AC-3        | easy       | 0.0105         | True   |
| 1 | 9         | AC3+MRV+LCV | easy       | 0.0834         | True,  |
| ( | Grid Size | Heuristic   | Difficulty | Time Taken (s) | Solved |
| 0 | 9         | AC-3        | medium     | 0.0197         | True   |
| 1 | 9         | AC3+MRV+LCV | medium     | 0.1924         | True,  |
| ( | Grid Size | Heuristic   | Difficulty | Time Taken (s) | Solved |
| 0 | 9         | AC-3        | hard       | 0.0356         | True   |
| 1 | 9         | AC3+MRV+LCV | hard       | 7.2434         | True,  |

*Figure 3.2*

# Chapter 4

## Conclusion

### 4.1 Results

The findings obtained from this project will demonstrate some of the comparative effectiveness of certain techniques to solve Sudoku puzzles with varying levels of difficulty. From the analysis that we carried out, the MRV heuristic and the AC-3 algorithm were validated as efficient. While time complexity and steps for easy and medium-level puzzles favored MRV for ease of application, for hard puzzles, the AC-3 algorithm was validated as the best solution, often proving itself as being a good start for solving its complex constraints. The test results also showed that utilizing some heuristics, such as MRV and LCV with the AC-3 algorithm, had a reverse effect on performance particularly for the easier level. Degree heuristics also failed miserably for complicated puzzles affirming the importance of using techniques that align with the complexity of the problem. The overall conclusion defines a strong foundation whereby MRV will be adopted for simpler puzzles and AC-3 would cater for the more complicated ones thereby striking a balance to form an efficient sudoku solver.

### 4.2 Conclusion

Thus, the now-concluded option adds that, among the many methods of solving Sudoku puzzles of varying difficulty level, MRV heuristic and AC-3 algorithm are the most effective methods of solving. This is possible because MRV heuristics work unbelievably well in the simpler puzzles, effectively lowering the computational time and complexity. Meanwhile, AC-3 algorithm seems to be more feasible for tough puzzles, being able to deal with quite a huge number of constraint satisfaction challenges. These techniques not only demonstrate the power of CSP in tackling logic-based problems, but they also pave the way toward the efficient hybrid solutions of other optimization challenges.

## 4.3 Future Work

Future work could be directed toward hybrid methods developed combining advantages of MRV and AC-3 by limiting their weaknesses in particular instances. The use of advanced AI methods, such as machine learning and neural networks, could be employed to further optimize the adaptation and efficiency of the solver. Knowledge of relevant scaling issues regarding larger grid sizes and more complicated constraints (such as Sudoku variants like Samurai Sudoku) would also be a worthwhile pursuit. Lastly, the performance of these algorithms would be greatly improved with implementations of parallel or distributed versions, enabling real-time solutions to even more difficult puzzles.

# References

- Darkeclipz. (2020). *Sudoku as a CSP*. GitHub repository. Retrieved from <https://github.com/darkeclipz/sudoku-csp>
- Grinberg, I. (2009). *Sudoku as a CSP*. CodeProject. Retrieved from <https://www.codeproject.com/Articles/34403/Sudoku-as-a-CSP>
- Li, D. (2019). *CSPSudokuSolver*. GitHub repository. Retrieved from <https://github.com/douglassli/CSPSudokuSolver>
- Parkude, P. (2020). *Formulation of CSP Problem: Sudoku Puzzle*. Medium. Retrieved from <https://medium.com/@co.2020.prkude/formulation-of-csp-problem-sudoku-puzzle-7d5e1d547382>
- Sandipan. (2017). *Solving Sudoku as a Constraint Satisfaction Problem*. SandipanWeb. Retrieved from <https://sandipanweb.wordpress.com/2017/03/17/solving-sudoku-as-a-constraint-satisfaction-problem-using-constraint-propagation-with-arc-consistency-checking-and-then-backtracking-with-minimum-remaning-value-heuristic-and-forward-checking/>
- Sudoku.com. (n.d.). *The History of Sudoku*. Retrieved from <https://sudoku.com/how-to-play/the-history-of-sudoku/>
- Crawford, P., & Gervet, C. (2009). *\*Solving Sudoku with constraint programming\**. ResearchGate. Retrieved from [https://www.researchgate.net/publication/226632124\\_Solving\\_Sudoku\\_with\\_Constraint\\_Programming](https://www.researchgate.net/publication/226632124_Solving_Sudoku_with_Constraint_Programming)
- Darkeclipz. (2020). *\*Sudoku as a CSP\**. GitHub repository. Retrieved from <https://github.com/darkeclipz/sudoku-csp>
- Grinberg, I. (2009). *\*Sudoku as a CSP\**. CodeProject. Retrieved from <https://www.codeproject.com/Articles/34403/Sudoku-as-a-CSP>
- Lewis, R. (2007). *\*On the combination of constraint programming and stochastic search: The Sudoku case\**. ResearchGate. Retrieved from [https://www.researchgate.net/publication/221411168\\_On\\_the\\_Combination\\_of\\_Constraint\\_Programming\\_and\\_Stochastic\\_Search\\_The\\_Sudoku\\_Case](https://www.researchgate.net/publication/221411168_On_the_Combination_of_Constraint_Programming_and_Stochastic_Search_The_Sudoku_Case)
- Li, D. (2019). *\*CSPSudokuSolver\**. GitHub repository. Retrieved from <https://github.com/douglassli/CSPSudokuSolver>
- Parkude, P. (2020). *\*Formulation of CSP Problem: Sudoku Puzzle\**. Medium. Retrieved from <https://medium.com/@co.2020.prkude/formulation-of-csp-problem-sudoku-puzzle-7d5e1d547382>

- Sandipan. (2017). \*Solving Sudoku as a Constraint Satisfaction Problem\*. SandipanWeb. Retrieved from <https://sandipanweb.wordpress.com/2017/03/17/solving-sudoku-as-a-constraint-satisfaction-problem-using-constraint-propagation-with-arc-consistency-checking-and-then-backtracking-with-minimum-remaining-value-heuristic-and-forward-checking/>
- Sudoku.com. (n.d.). \*The history of Sudoku\*. Retrieved from <https://sudoku.com/how-to-play/the-history-of-sudoku/>
- Christensen, J., & Emerson, T. (2006). \*Solving Sudoku with constraint satisfaction problems\*. Retrieved from <https://saveitforparts.wordpress.com/wp-content/uploads/2009/11/sudoku.pdf>
- Eck, A. (2021). \*CSCI364\_HW2: Constraint satisfaction problems and Sudoku\*. Oberlin College. Retrieved from [https://www.cs.oberlin.edu/~aeck/Fall2021/CSCI364/Assignments/HW2/CSCI364\\_HW2\\_Constraints.pdf](https://www.cs.oberlin.edu/~aeck/Fall2021/CSCI364/Assignments/HW2/CSCI364_HW2_Constraints.pdf)